

Relazione di Progetto: Sistema di Gestione Aula Studio

Traccia 2: Sistema di Gestione degli Accessi e delle Prenotazioni di un'Aula Studio
Domenico Gerardo Salvati - Mathias Pio Toscano

Ambiente di Sviluppo

Il progetto è stato sviluppato, testato e collaudato in ambiente Windows avvalendosi del sottosistema WSL (Windows Subsystem for Linux). L'orchestrazione della compilazione è stata gestita tramite un file Makefile strutturato con target multipli (make e make clean). Il codice è stato scritto rispettando rigorosamente lo standard C99 e compilato su GCC.

1. Motivazione della Scelta dell'ADT

Per la realizzazione del Gestore dell'Aula Studio, abbiamo analizzato a fondo i requisiti della traccia e optato per l'utilizzo di specifici Tipi di Dato Astratti (ADT). Questo ci ha permesso di gestire in modo efficiente i posti a sedere, la lista d'attesa e lo storico delle prenotazioni:

- **Array Dinamico (Posti a sedere):** I posti fisici all'interno dell'aula sono stati modellati tramite un array di puntatori a Studente, allocato dinamicamente in base alla capienza inserita all'avvio del programma. Questa scelta ci garantisce un accesso diretto all'indice del posto in tempo costante $O(1)$, semplificando notevolmente le operazioni di assegnazione e verifica.
- **Tabella Hash (Ricerca e Tracciamento):** Per verificare in tempo reale chi è fisicamente presente in aula ed evitare ingressi duplicati della stessa matricola, avevamo bisogno di ricerche estremamente veloci. Abbiamo implementato una Tabella Hash basata sull'algoritmo *djb2*. Per gestire le collisioni in modo sicuro abbiamo usato la tecnica del *Chaining* (concatenamento tramite liste collegate). In questo modo, l'ingresso e il check-out avvengono con una complessità temporale media pari a $O(1)$.
- **Coda FIFO (Lista d'attesa):** Gli studenti che tentano l'accesso quando l'aula è satura vengono inseriti in una Coda. Trattandosi di una struttura *First-In, First-Out*, garantisce in modo naturale il principio di equità richiesto dal problema: il primo studente a essere messo in attesa sarà il primo a subentrare automaticamente quando verrà registrata un'uscita.
- **Lista Concatenata (Registro Prenotazioni):** Lo storico delle pratiche di prenotazione è

memorizzato in una lista semplicemente concatenata. Poiché il numero di prenotazioni future non è noto a priori, questa struttura ci offre flessibilità dimensionale ed evita le onerose operazioni di riallocazione della memoria (realloc).

2. Progettazione e Architettura del Sistema

Il codice è stato suddiviso in moduli software indipendenti (file .h e .c) per garantire l'incapsulamento dei dati (*information hiding*) e una netta separazione delle responsabilità logiche:

- **Modulo Studente:** Gestisce l'anagrafica utente. I campi interni sono nascosti al main tramite un puntatore opaco a struct. Utilizza funzioni sicure come strncpy in fase di inizializzazione per prevenire vulnerabilità di buffer overflow.
- **Modulo Prenotazione:** Associa un'istanza Studente a una data, una fascia oraria e un posto specifico. Espone funzioni interne per validare formalmente e logicamente le stringhe di input (verificando formati e coerenza temporale/calendario).
- **Modulo Coda e Item:** La lista d'attesa è stata implementata introducendo un livello di astrazione tramite il file item.h (typedef Studente item;). Questo rende l'ADT Coda completamente generico e riutilizzabile: modificando un solo file, la struttura potrebbe accodare qualsiasi altro tipo di dato in futuri ampliamenti.
- **Modulo Aula:** Rappresenta il nucleo centrale logico del sistema. Coordina e orchestra tutte le strutture dati sottostanti (Hash, Coda, Array, Registro). Implementa la logica, valutando matematicamente le intersezioni tra fasce orarie per prevenire doppie prenotazioni.
- **Interfaccia Utente (Main):** Gestisce l'interazione a riga di comando (CLI). Cattura l'input dell'utente validandolo sistematicamente tramite la pulizia del buffer (getchar()), prevenendo loop infiniti o crash dovuti a digitazioni anomale.
- **Suite di Test (Main Test):** Modulo eseguibile isolato per la verifica automatizzata di tutte le strutture dati.

3. Come Usare e Interagire con il Sistema

Il software è interamente guidato da un'interfaccia a riga di comando (CLI) facile ed intuitiva. Per avviare il sistema occorre aprire il terminale nella root del progetto ed eseguire:

1. make per compilare il codice sorgente.
2. ./gestore_aula per eseguire l'applicativo.

All'avvio, il programma inizializza un'aula da 50 posti e presenta un menù principale numerato. L'utente digita il numero corrispondente all'azione desiderata (Ingresso diretto, Uscita, o accesso al Sottomenù Prenotazioni) e preme Invio. Il software guida l'inserimento richiedendo i

dati necessari e avvisando l'utente dei formati richiesti (es. Date in "GG/MM/AAAA" e orari in "HH:MM-HH:MM").

Il sistema stampa stringhe formattate (es. >>> SUCCESSO, >>> ERRORE o >>> AVVISO CRONOLOGICO) per confermare l'esito dell'operazione. È presente l'opzione "3" nel menù per generare in qualsiasi momento una fotografia in tempo reale delle occupazioni fisiche e della lista d'attesa, e l'opzione "4" nel sottomenù per generare il report statistico completo accessi/assenze.

4. Specifica Sintattica e Semantica

Di seguito è documentata rigorosamente l'intera architettura esposta dalle interfacce (file .h) dei quattro Moduli Principali, per un totale di 32 funzioni implementate nel progetto.

4.1 Modulo Studente (studente.h)

- **Studente studente_crea(char* matricola, char* nome, char* corso);**
Semantica: Alloca dinamicamente e inizializza un nuovo studente.
Pre-condizioni: I parametri non devono essere NULL.
Post-condizioni: Creazione di un oggetto Studente con copie esatte delle stringhe.
Ritorno: Puntatore allo Studente creato, o NULL in caso di fallimento.
- **void studente_distruggi(Studente* s);**
Semantica: Libera la memoria dinamica dell'oggetto.
Pre-condizioni: Puntatore valido e allocato.
Post-condizioni: Memoria liberata, puntatore azzerato a NULL per prevenire dangling pointer.
- **char* studente_ottieni_matricola(Studente s);**
Semantica: Fornisce l'accesso in sola lettura alla matricola.
Pre-condizioni: Oggetto Studente valido.
Ritorno: Puntatore alla stringa della matricola.
- **void studente_stampa(Studente s);**
Semantica: Stampa a video i dettagli anagrafici formattati.
Pre-condizioni: Oggetto Studente valido.
Post-condizioni: Nessuna modifica di stato interna.

4.2 Modulo Prenotazione (prenotazione.h)

- **Prenotazione prenotazione_crea(Studente s, char* data, char* fascia, int posto);**
Semantica: Alloca e popola i campi temporali e spaziali della pratica.
Pre-condizioni: Parametri e oggetto Studente validi.

- Ritorno:** Puntatore alla Prenotazione creata.
- **void prenotazione_distruggi(Prenotazione* p);**
Semantica: Dealloca l'oggetto Prenotazione dalla memoria.
Pre-condizioni: Puntatore valido. Lo studente associato NON viene distrutto.
 - **Studiante prenotazione_ottieni_studiante(Prenotazione p);**
Semantica: Restituisce il riferimento allo studente associato.
Ritorno: Puntatore all'oggetto Studente.
 - **void prenotazione_effettua_checkin(Prenotazione p);**
Semantica: Aggiorna lo stato confermando l'ingresso in aula.
Post-condizioni: Lo stato interno della pratica passa a 1.
 - **int prenotazione_ottieni_stato(Prenotazione p);**
Semantica: Legge lo stato attuale della pratica.
Ritorno: 0 (Solo prenotato), 1 (Check-in fatto).
 - **int prenotazione_ottieni_posto(Prenotazione p);**
Semantica: Restituisce l'indice della sedia riservata.
Ritorno: Intero del posto.
 - **char* prenotazione_ottieni_data(Prenotazione p);**
Semantica: Restituisce la stringa contenente la data.
 - **char* prenotazione_ottieni_fascia(Prenotazione p);**
Semantica: Restituisce la stringa contenente l'orario.
 - **void prenotazione_stampa(Prenotazione p);**
Semantica: Output a terminale dei dettagli formattati della pratica.
 - **int valida_data(const char* data);**
Semantica: Esegue un controllo per verificare il formato GG/MM/AAAA e la coerenza del calendario (bisestili inclusi).
Ritorno: 1 (valido), 0 (invalido).
 - **int valida_fascia(const char* fascia);**
Semantica: Verifica il formato HH:MM-HH:MM e assicura che l'orario di uscita sia matematicamente successivo a quello d'ingresso.

4.3 Modulo Coda (coda.h)

- **Coda coda_crea(void);**
Semantica: Alloca e inizializza una nuova lista d'attesa (FIFO).
Ritorno: Puntatore alla Coda.
- **int coda_vuota(Coda q);**
Semantica: Verifica l'assenza di elementi.
Ritorno: 1 (vuota), 0 (piena).
- **int coda_inserisci(item val, Coda q);**

Semantica: Accoda in modo dinamico un nuovo elemento sul fondo.

Post-condizioni: Il contatore viene incrementato e il nuovo nodo diventa la coda della lista.

- **item coda_estrai(Coda q);**

Semantica: Estrae la testa della coda garantendo il principio FIFO.

Ritorno: L'elemento 'item' rimosso, deallocando il nodo strutturale associato.

- **void coda_distruggi(Coda* q);**

Semantica: Svuota la memoria dinamica della struttura.

- **void coda_stampa(Coda q);**

Semantica: Scorre iterativamente l'attesa stampando i componenti.

- **int coda_lunghezza(Coda q);**

Semantica: Restituisce il conteggio in tempo $O(1)$ degli elementi in coda.

4.4 Modulo Aula (aula.h)

- **Aula aula_crea(char* nome, int capienza_massima);**

Semantica: Coordina la creazione dell'ecosistema aula (Hash, Array Posti, Coda, Storico).

Post-condizioni: Strutture dati allocate e pronte. Ritorna il tipo Aula.

- **void aula_distruggi(Aula* a);**

Semantica: Distrugge le allocazioni per prevenire leak.

- **int aula_ingresso(Aula a, Studente s, int posto_richiesto);**

Semantica: Smista l'utente. Se c'è posto, mette lo studente nell'array e aggiunge la chiave nell'Hash. Se è pieno, usa l'ADT Coda.

Ritorno: 1 (Seduto), 2 (Accodato), 0 (Errore).

- **int aula_uscita(Aula a, char* matricola_da_cercare);**

Semantica: Rimuove dall'Hash e dall'array in tempo rapido, innescando automaticamente il subentro (dequeue) se la coda non è vuota.

- **void aula_stampa_stato(Aula a);**

Semantica: Stampa a schermo le occupazioni e attese.

- **int aula_aggiungi_prenotazione(Aula a, Prenotazione p);**

Semantica: Algoritmo di prevenzione: itera sullo storico controllando le collisioni di fascia oraria per il posto richiesto e data prima di confermare.

- **int aula_checkin_prenotazione(Aula a, char* matricola);**

Semantica: Trasforma la prenotazione in ingresso reale, evitando check-in duplicati e smistando l'utente.

- **int aula annulla_prenotazione(Aula a, char* matricola);**

Semantica: Estrae il nodo dallo storico e svuota la memoria pendente.

- **void aula_stampa_report_prenotazioni(Aula a);**

Semantica: Stampa il report delle prenotazioni.

- **int aula_studente_esiste(Aula a, char* matricola);**

Semantica: Previene ingressi multipli della stessa persona sondando preventivamente la Tabella Hash.

5. Razionale dei Casi di Test

La fase di validazione del codice è stata completata tramite un modulo eseguibile automatizzato (main_test.c). Il sistema isola ogni scenario applicativo all'interno di funzioni void dedicate, valutando gli stati in memoria tramite standard assert(). La test suite mappa puntualmente tutti gli 8 requisiti obbligatori specificati dalla Traccia 2:

1. Verifica della registrazione degli studenti (test_registrazione_studenti)

- **Obiettivo:** Validare l'allocazione dinamica e il completamento dei dati anagrafici.
- **Input:** Chiamata al costruttore passandogli "MAT01", "Mario", "Informatica".
- **Oracolo:** Il costruttore deve rilasciare l'oggetto senza generare puntatori nulli.
- **Output:** Assert su `s != NULL` superato con successo.

2. Verifica dell'ingresso senza prenotazione (test_ingresso_senza_prenotazione)

- **Obiettivo:** Verificare l'assegnazione del posto diretta.
- **Input:** Richiesta per sedersi al posto fisico 0.
- **Oracolo:** La funzione deve marcare l'array interno e ritornare 1 (successo).
- **Output:** Lo stato dell'aula viene alterato, la funzione passa l'assert a 1.

3. Test della lista di attesa (test_lista_attesa)

- **Obiettivo:** Stress test della saturazione per innescare l'ADT Coda.
- **Input:** Si forza un'aula di capienza 1 inserendo un secondo studente nel posto già occupato.
- **Oracolo:** L'algoritmo non deve sovrascrivere l'array, ma redirigere il flusso verso la coda restituendo 2.
- **Output:** Inserimento nella struttura FIFO convalidato.

4. Test dell'inserimento delle prenotazioni (test_inserimento_prenotazioni)

- **Obiettivo:** Archiviare una pratica all'interno della lista concatenata (registro).
- **Input:** Prenotazione lecita di Mario al posto 2.
- **Oracolo:** Modifica dei puntatori testa del registro con restituzione del flag 1.
- **Output:** Oggetto immagazzinato correttamente nella memoria storica.

5. **Verifica disponibilità e blocco conflitti (test_disponibilita_posti)**

- **Obiettivo:** Test di robustezza della logica matematica.
- **Input:** Si tentano di registrare due prenotazioni sullo stesso posto fisico con le fasce "10:00-12:00" e "11:00-13:00".
- **Oracolo:** La prima passa (1), la seconda deve subire un blocco (0) in virtù dell'intersezione.
- **Output:** Risoluzione algebrica riuscita. Prenotazione non andata a buon fine.

6. **Verifica annullamento e aggiornamento (test annullamento_disponibilita)**

- **Obiettivo:** Controllare la deallocazione di un nodo della lista concatenata.
- **Input:** Comando di annullamento pratica indicando la matricola chiave "MAT08".
- **Oracolo:** L'algoritmo deve intercettare il nodo, toglierlo dalla catena, invocare free() e tornare 1.
- **Output:** Nessun dangling pointer generato, rimozione completata.

7. **Test del check-in e check-out (test_checkin_checkout)**

- **Obiettivo:** Validare la transizione di stato vitale di una risorsa (prenotazione -> convalida -> uscita).
- **Input:** Operazione sequenziale di Check-in e successiva Uscita per "MAT09".
- **Oracolo:** Entrambe le chiamate devono interagire e scalare la Tabella Hash in tempo reale (restituendo 1).
- **Output:** Ciclo completo simulato positivamente.

8. **Test dello storico e dei report (test_storico_report)**

- **Obiettivo:** Gestione dei casi limite e delle eccezioni a runtime.
- **Input:** Invocazione cicli di stampa su un'Aula completamente non popolata.
- **Oracolo:** Il sistema deve superare i controlli di guardia a NULL senza generare alcun *Segmentation Fault* esplorando memoria non assegnata.
- **Output:** Il report stampa i parametri a zero e il programma raggiunge la terminazione dell'eseguibile stabile.